Indian Used Car Price Prediction

Developed By

Rishabh Alpeshbhai Shah (23SS02IT181)

Project Abstract

This project aims to predict the price of used cars in Indian metro cities using Machine Learning techniques. The model analyzes various features such as company, model, variant, fuel type, kilometers driven, and quality score to estimate an accurate market price.

The goal is to build a smart and reliable pricing system that helps users make informed decisions while buying or selling used cars.

Problem Statement

To develop a reliable machine learning model that accurately predicts the market price of used cars in India based on key features like:

- Car make and model
- Age of the vehicle
- Kilometers driven
- Fuel type
- Ownership history

This system aims to ensure fair, transparent, and data-driven pricing in the used car market.

Dataset Overview

The project uses the **Indian IT Cities Used Car Dataset 2023**, which contains detailed information about used car listings across major metro cities in India.

Dataset Features Include:

- Car specifications
- Pricing details
- Dealer information
- Ownership history
- Warranty and quality scores

Data Dictionary

Column Name	Description
ID	Unique identifier for each listing
Company	Car manufacturer name
Model	Car model name
Variant	Variant of the car
Fuel Type	Type of fuel used
Color	Car color
Kilometers	Distance driven (in KM)
Body Style	Type of car body
Transmission	Transmission type
Manufacture Date	Date of manufacturing
Model Year	Year of the model
CNG Kit	Availability of CNG kit
Price	Selling price of the car
Owner Type	Number of previous owners
Dealer State	State of dealer
Dealer Name	Dealer name
City	City of listing
Warranty	Warranty provided
Quality Score	Overall quality rating

```
#Indian Used Car Price Predictor
```

```
#Importing the required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
#Loading the dataset
df = pd.read_csv('usedCars.csv')
df.head()
```

1 to 5 of 5 entries Filter

index	Id	Company	Model	Variant	FuelType	Colour	Kilometer	BodyStyle	TransmissionType	ManufactureDate	ModelYear	CngKit	Price	Owner	DealerState	DealerName	City	Warranty	QualityScore
0	555675	MARUTI SUZUKI	CELERIO(2017-2019)	1.0 ZXI AMT O	PETROL	Silver	33197	HATCHBACK	NaN	2018-02-01	2018	NaN	5.75 Lakhs	1st Owner	Karnataka	Top Gear Cars	Bangalore	1	7.8
1	556383	MARUTI SUZUKI	ALTO	LXI	PETROL	Red	10322	HATCHBACK	Manual	2021-03-01	2021	NaN	4.35 Lakhs	1st Owner	Karnataka	Renew 4 u Automobiles PVT Ltd	Bangalore	1	8.3
2	556422	HYUNDAI	GRAND I10	1.2 KAPPA ASTA	PETROL	Grey	37889	HATCHBACK	Manual	2015-03-01	2015	NaN	4.7 Lakhs	1st Owner	Karnataka	Anant Cars Auto Pvt Ltd	Bangalore	1	7.9
3	556771	TATA	NEXON	XT PLUS	PETROL	A Blue	13106	HATCHBACK	NaN	2020-08-01	2020	NaN	9.9 Lakhs	1st Owner	Karnataka	Adeep Motors	Bangalore	1	8.1
4	559619	FORD	FIGO	EXI DURATORQ 1.4	DIESEL	Silver	104614	HATCHBACK	Manual	2010-11-01	2010	NaN	2.7 Lakhs	2nd Owner	Karnataka	Zippy Automart	Bangalore	0	7.5

Show 25 per page
Like what you see? Visit the data table notebook to learn more about interactive tables.

Next steps: Generate code with df New interactive sheet

Data Preprocessing Part 1

```
#Shape of the dataset
df.shape
```

```
(1064, 19)
```

```
#colums in the dataset
df.columns
```

```
Index(['Id', 'Company', 'Model', 'Variant', 'FuelType', 'Colour', 'Kilometer',
       'BodyStyle', 'TransmissionType', 'ManufactureDate', 'ModelYear',
       'CngKit', 'Price', 'Owner', 'DealerState', 'DealerName', 'City',
       'Warranty', 'QualityScore'],
      dtype='object')
```

```
#dropping column ID, as it is a identifier and not required for analysis
df.drop('Id',axis=1,inplace=True)
```

```
#Column Data Types
df.dtypes
```

	0
Company	object
Model	object
Variant	object
FuelType	object
Colour	object
Kilometer	int64
BodyStyle	object
TransmissionType	object
ManufactureDate	object
ModelYear	int64
CngKit	object
Price	object
Owner	object
DealerState	object
DealerName	object
City	object
Warranty	int64
QualityScore	float64

dtype: object

```
def convert_amount(amount_str):
    if pd.isna(amount_str):
        return None

    amount_str = str(amount_str)

    if "Lakhs" in amount_str:
        return float(amount_str.replace(' Lakhs', '').replace(',', '')) * 100000
    elif "Crore" in amount_str:
        return float(amount_str.replace(' Crore', '').replace(',', '')) * 10000000
    else:
        return float(amount_str.replace(',', ''))

df['Price'] = df['Price'].apply(convert_amount)
```

```
[13]
#Checking for null values percentage wise
df.isnull().sum()/df.shape[0]*100
```

	0
Company	0.000000
Model	0.000000
Variant	0.000000
FuelType	0.093985
Colour	0.000000
Kilometer	0.000000
BodyStyle	0.000000
TransmissionType	67.105263
ManufactureDate	0.000000
ModelYear	0.000000
CngKit	97.932331
Price	0.000000
Owner	0.000000
DealerState	0.000000
DealerName	0.000000
City	0.000000
Warranty	0.000000
QualityScore	0.000000

dtype: float64

In the dataset, three columns contain missing values: **FuelType, TransmissionType, and CngKit**. Appropriate data preprocessing steps were applied to handle these missing values effectively:

- **CngKit:**
  The majority of cars in the dataset do not use CNG, and vehicles with CNG can already be identified using the *FuelType* column. Therefore, the **CngKit column was removed** as it provides redundant information and has limited impact on the analysis.

- **TransmissionType:**
  Approximately **67% of the values are missing** in this column. Due to the high proportion of missing data, it is not reliable for analysis. Hence, the **TransmissionType column was dropped** from the dataset.

- **FuelType:**
  Since *FuelType* is an important feature for price prediction, rows with missing values in this column were **removed** to maintain data quality and model accuracy.

```
[14]
df.drop('CngKit', axis=1, inplace=True)
```

```
[15]
#Dropping TransmissionType column
df.drop('TransmissionType',axis=1,inplace=True)
```

```
[16]
#Removing null values from FuelType column
df['FuelType'].dropna(inplace=True)
```

Manufacture Date:
The *Manufacture Date* column was removed from the dataset as it provides information about the age of the vehicle, which is already captured by the *Model Year* column.

Since both features convey similar information, retaining both would introduce redundancy. Therefore, the *Manufacture Date* column was dropped to simplify the dataset and improve model efficiency.

```
[17]
df.drop('ManufactureDate', axis = 1, inplace=True)
```

```
[18]
df.drop('Variant', axis = 1, inplace=True)
```

Converted the **Model Year** column into **Car Age** for better representation of the vehicle's depreciation.

```
[19]
df['ModelYear'] = 2023 - df['ModelYear']
df.rename(columns={'ModelYear':'Age'},inplace=True)
```

```
for i in df.columns:
    print(i, df[i].nunique())
```

```
Company 23
Model 218
FuelType 5
Colour 76
Kilometer 1006
BodyStyle 10
Age 17
Price 362
Owner 4
DealerState 10
DealerName 57
City 11
Warranty 2
QualityScore 43
```

Descriptive Statistics

```
df.describe()
```

1 to 8 of 8 entries Filter

index	Kilometer	Age	Price	Warranty	QualityScore
count	1064.0	1064.0	1064.0	1064.0	1064.0
mean	52807.18796992481	6.135338345864661	835053.5714285715	0.7387218045112782	7.770206766917293
std	33840.29697854122	2.9967858728054546	572653.7833167345	0.4395377945793955	0.7197166491359881
min	101.0	0.0	95000.0	0.0	0.0
25%	32113.5	4.0	484999.99999999994	0.0	7.5
50%	49432.0	6.0	675000.0	1.0	7.8
75%	68828.5	8.0	985000.0	1.0	8.1
max	640000.0	20.0	8500000.0	1.0	9.4

Show 25 per page
Like what you see? Visit the data table notebook to learn more about interactive tables.

```
df.head()
```

1 to 5 of 5 entries Filter

index	Company	Model	FuelType	Colour	Kilometer	BodyStyle	Age	Price	Owner	DealerState	DealerName	City	Warranty	QualityScore
0	MARUTI SUZUKI	CELERIO(2017-2019)	PETROL	Silver	33197	HATCHBACK	5	575000.0	1st Owner	Karnataka	Top Gear Cars	Bangalore	1	7.8
1	MARUTI SUZUKI	ALTO	PETROL	Red	10322	HATCHBACK	2	434999.99999999994	1st Owner	Karnataka	Renew 4 u Automobiles PVT Ltd	Bangalore	1	8.3
2	HYUNDAI	GRAND I10	PETROL	Grey	37889	HATCHBACK	8	470000.0	1st Owner	Karnataka	Anant Cars Auto Pvt Ltd	Bangalore	1	7.9
3	TATA	NEXON	PETROL	A Blue	13106	HATCHBACK	3	990000.0	1st Owner	Karnataka	Adeep Motors	Bangalore	1	8.1
4	FORD	FIGO	DIESEL	Silver	104614	HATCHBACK	13	270000.0	2nd Owner	Karnataka	Zippy Automart	Bangalore	0	7.5

Show 25 per page
Like what you see? Visit the data table notebook to learn more about interactive tables.

Next steps: Generate code with df New interactive sheet

Exploratory Data Analysis (EDA)

In the exploratory data analysis phase, the distribution of data across all columns is examined to gain a better understanding of the dataset. This includes identifying patterns, trends, and any anomalies present in the data.

Further, the relationship between the target variable (*Price*) and the independent variables is analyzed to determine which features significantly influence car pricing.

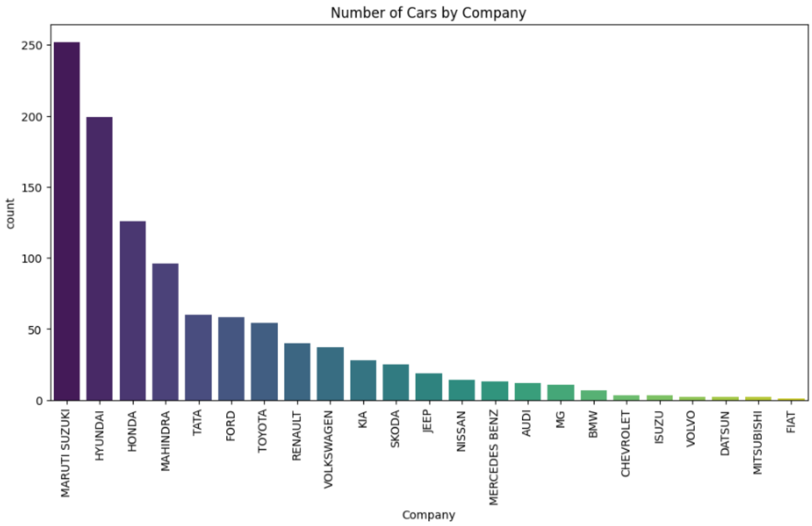
Car Company Analysis

From the graph, we can observe the distribution of used cars across different companies in the dataset. There are a total of **23 car companies** represented.

Among them, **Maruti Suzuki, Hyundai, Honda, Mahindra, and Tata** have the highest number of listings in the dataset. This indicates that these brands are more commonly available in the used car market.

```
[27]
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12,6))
sns.countplot(x=df['Company'],
            order=df['Company'].value_counts().index,
            palette='viridis')

plt.xticks(rotation=90)
plt.title("Number of Cars by Company")
plt.show()
```

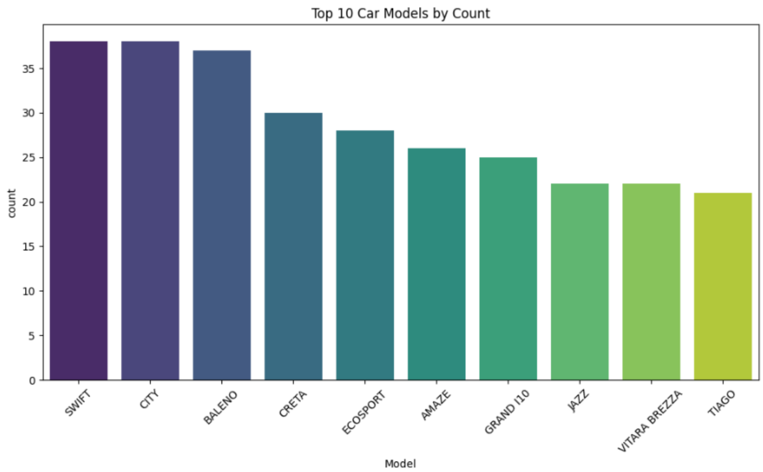
```
/tmp/ipykernel_3963/1172234412.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.countplot(x=df['Company'],
```



The higher number of listings may be due to their **popularity, larger market share, and widespread usage** in India. However, this does not necessarily imply durability or resale value, but it highlights their strong presence in the market.

```
[31]
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12,6))

sns.countplot(x=df['Model'],
            order=df['Model'].value_counts().iloc[:10].index,
            palette='viridis')

plt.xticks(rotation=45)
plt.title("Top 10 Car Models by Count")
plt.show()
```

```
/tmp/ipykernel_3963/3991792832.py:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.countplot(x=df['Model'],
```



Top Car Models Analysis

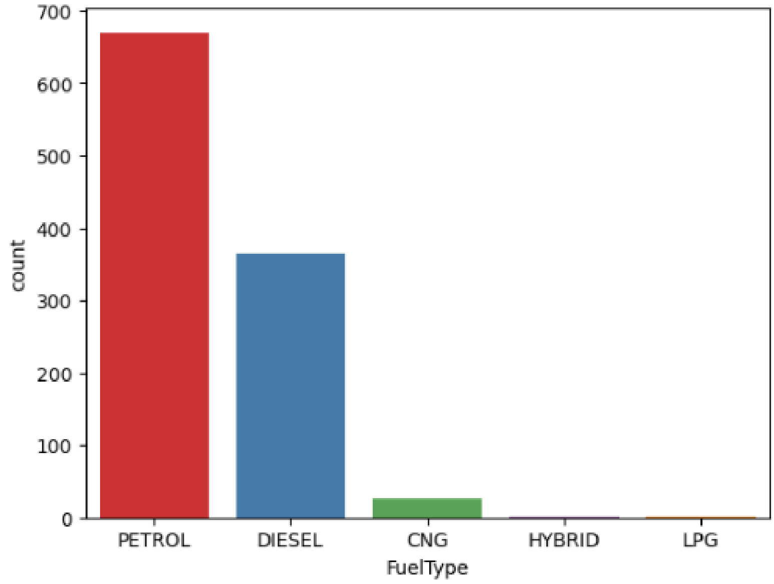
From the graph, we can observe the top 10 most frequently listed car models in the dataset. **Swift and Honda City** have the highest number of listings, followed by **Baleno, Creta, and EcoSport**.

This indicates that these models are **more commonly available** in the used car market. Their high frequency may be due to their **popularity, widespread ownership, and strong market presence** in India.

Additionally, the higher number of listings for models like **Swift and Honda City** may also suggest **higher demand and active resale activity** in the used car segment. However, this does not directly imply durability or resale value, but rather reflects their popularity and circulation in the market.

```
[33]
sns.countplot(x='FuelType', data=df, hue='FuelType', palette='Set1', legend=False)
```

```
<Axes: xlabel='FuelType', ylabel='count'>
```



Fuel Type Analysis

From the graph, it is observed that the majority of used cars available for resale have a **petrol engine**, with more than 650 listings, followed by around 350 cars with **diesel engines**.

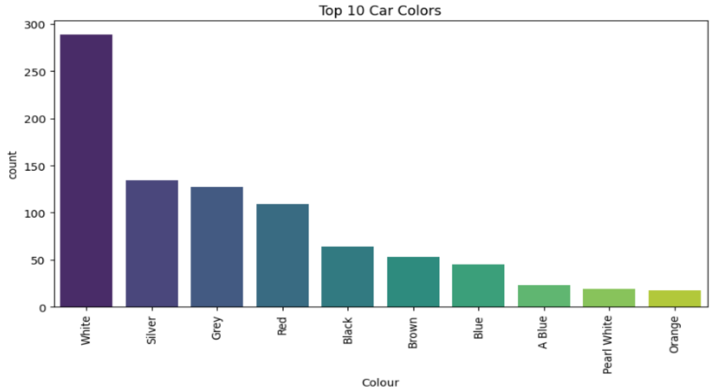
A very small number of cars run on **CNG**, while **hybrid and LPG vehicles** are present in negligible quantities.

This distribution indicates that **petrol and diesel cars dominate the used car market**, likely due to their widespread usage and availability. It also suggests higher resale activity for these fuel types compared to others.

[34]

```python
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10,5))

sns.countplot(x='Colour',
              data=df,
              order=df['Colour'].value_counts().iloc[:10].index,
              palette='viridis')

plt.xticks(rotation=90)
plt.title("Top 10 Car Colors")
plt.show()
```

```
/tmp/ipykernel_5759/1543033074.py:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.countplot(x='Colour',
```



Car Color Analysis

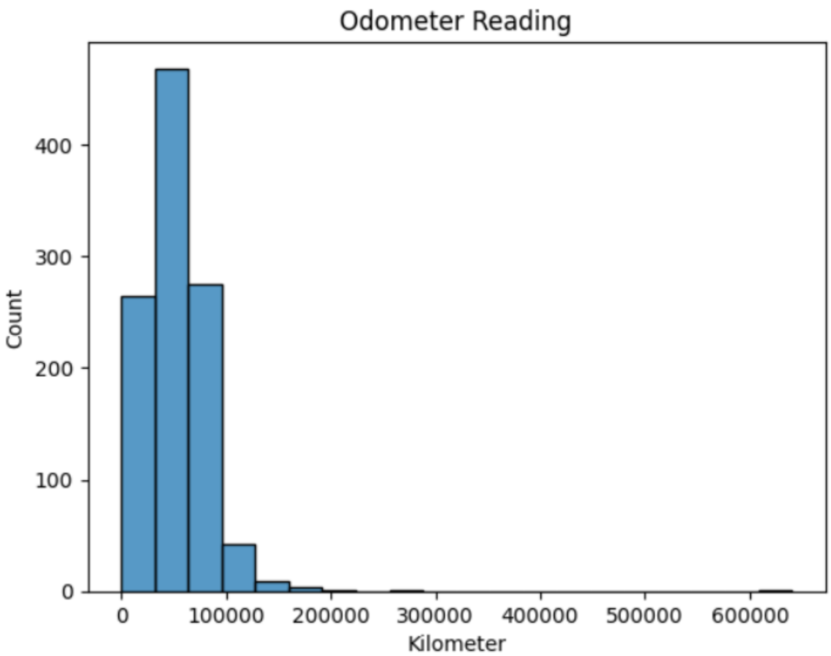
Although the color of a car does not affect its performance, it can influence buyer preferences in the used car market. From the graph, **white** is the most common color among listed cars, followed by **silver, grey, red, and black**.

This indicates that these colors are more widely available and preferred in the market. However, the higher frequency of these colors reflects **popularity and availability**, and does not necessarily guarantee higher resale value.

[35]

```python
#Odometer reading distribution
sns.histplot(x = 'Kilometer', data = df, bins = 20).set_title('Odometer Reading')
```

```
Text(0.5, 1.0, 'Odometer Reading')
```



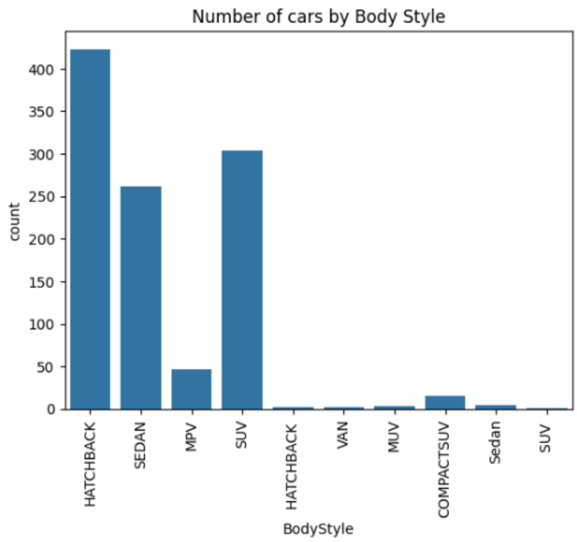
Odometer (Kilometers Driven) Analysis

This graph shows the distribution of kilometers driven for cars in the dataset. Most vehicles have an odometer reading **below 100,000 km**, with a large concentration in the **30,000–50,000 km** range.

This indicates that the dataset is dominated by **moderately used cars**, which are commonly listed in the resale market. While lower mileage is generally associated with better condition, the graph primarily reflects **availability**, not necessarily demand or resale value.

```
[36]
#Body style count
sns.countplot(x = 'BodyStyle', data = df).set_title('Number of cars by Body Style')
plt.xticks(rotation = 90)
```

```
([0, 1, 2, 3, 4, 5, 6, 7, 8, 9],
 [Text(0, 0, 'HATCHBACK'),
  Text(1, 0, 'SEDAN'),
  Text(2, 0, 'MPV'),
  Text(3, 0, 'SUV'),
  Text(4, 0, 'HATCHBACK '),
  Text(5, 0, 'VAN'),
  Text(6, 0, 'MUV'),
  Text(7, 0, 'COMPACTSUV'),
  Text(8, 0, 'Sedan'),
  Text(9, 0, 'SUV ')])
```



Body Style Analysis

According to the graph, most cars in the dataset belong to the **Hatchback, SUV, and Sedan** body styles. This indicates that these categories are **more commonly available** in the used car market.

Their higher frequency likely reflects their **popularity and widespread usage**. However, this distribution mainly shows **availability**, and does not directly imply higher demand or resale value.

Car Age Analysis

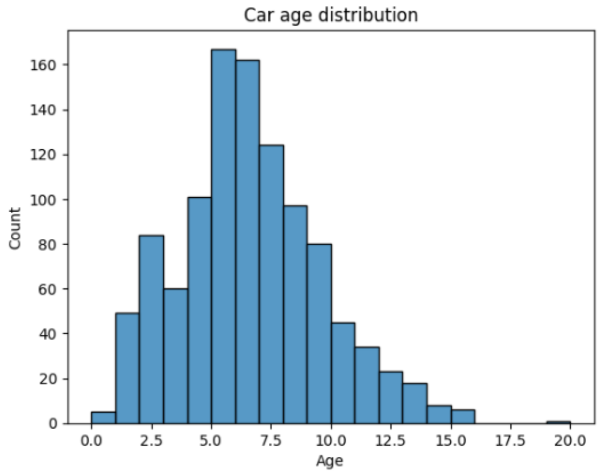
The age of a car is an important factor influencing its resale value. From the dataset, most cars fall in the **5–7 years** age range, and a large proportion of vehicles are **older than 5 years**.

This indicates that the dataset is dominated by **moderately aged cars**, which may generally have lower prices due to depreciation. However, there is also a noticeable number of cars with **less than 5 years of age**, which are likely to retain relatively higher value.

Additionally, there is a car with an age close to **20 years**, which can be considered a potential **outlier** in the dataset.

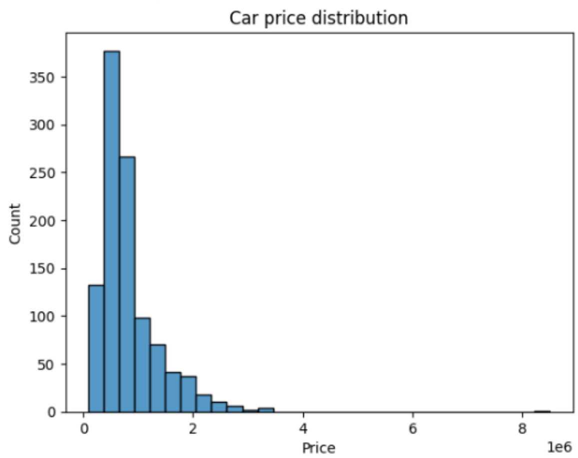
```
[37]
#Car age distribution
sns.histplot(x = 'Age', data = df, bins = 20).set_title('Car age distribution')
```

```
Text(0.5, 1.0, 'Car age distribution')
```




```
[38]
#Price distribution
sns.histplot(x = 'Price', data =df, bins = 30).set_title('Car price distribution')
```

```
Text(0.5, 1.0, 'Car price distribution')
```



Price Distribution Analysis

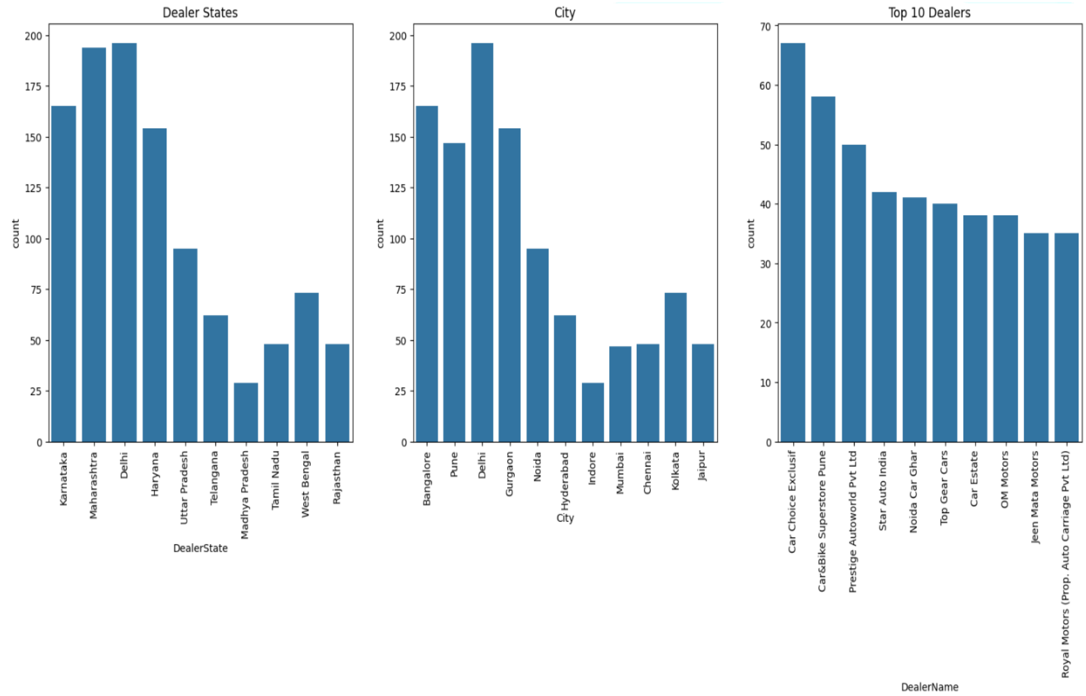
This graph shows the distribution of car prices in the dataset. Most cars are priced between **₹3 lakhs to ₹9 lakhs**, with the highest concentration in the **₹3 lakhs to ₹6 lakhs** range.

This indicates that the dataset is dominated by **mid-range priced vehicles**, which are commonly available in the used car market.

Additionally, a few cars have prices exceeding **₹20 lakhs**, which may represent **premium or luxury vehicles**, or could be potential **outliers**.

```
[39]
fig, ax = plt.subplots(1,3,figsize=(20,7))

#Dealer State
sns.countplot(x = 'DealerState', data = df, ax = ax[0]).set_title('Dealer States')
ax[0].set_xticklabels(ax[0].get_xticklabels(), rotation = 90)

#City
sns.countplot(x = 'City', data = df, ax = ax[1]).set_title('City')
ax[1].set_xticklabels(ax[1].get_xticklabels(), rotation = 90)

#top 10 dealers
sns.countplot(x = 'DealerName', data = df, order = df['DealerName'].value_counts().iloc[:10].index, ax = ax[2]).set_title('Top 10 Dealers')
ax[2].set_xticklabels(ax[2].get_xticklabels(), rotation = 90)
```

```
/tmp/ipykernel_5759/196354263.py:5: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax[0].set_xticklabels(ax[0].get_xticklabels(), rotation = 90)
/tmp/ipykernel_5759/196354263.py:9: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax[1].set_xticklabels(ax[1].get_xticklabels(), rotation = 90)
/tmp/ipykernel_5759/196354263.py:13: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax[2].set_xticklabels(ax[2].get_xticklabels(), rotation = 90)
[Text(0, 0, 'Car Choice Exclusif'),
 Text(1, 0, 'Car&Bike Superstore Pune'),
 Text(2, 0, 'Prestige Autoworld Pvt Ltd'),
 Text(3, 0, 'Star Auto India'),
 Text(4, 0, 'Noida Car Ghar'),
 Text(5, 0, 'Top Gear Cars'),
 Text(6, 0, 'Car Estate'),
 Text(7, 0, 'OM Motors'),
 Text(8, 0, 'Jeen Mata Motors'),
 Text(9, 0, 'Royal Motors (Prop. Auto Carriage Pvt Ltd)')]
```



Dealer Distribution Analysis

These graphs show the distribution of cars based on **dealer state, city, and dealer name**.

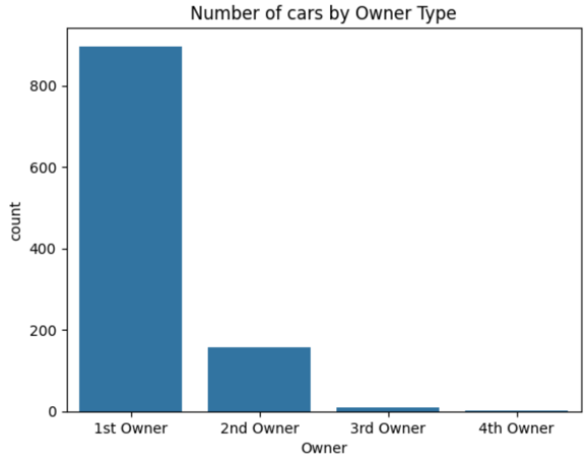
- **Dealer State:**
  Delhi and Maharashtra have the highest number of used car listings, followed by Karnataka and Haryana. This indicates a higher concentration of resale activity in these regions.

- **Dealer City:**
  Delhi has the highest number of cars, which aligns with the state-level observation. However, Bangalore shows more listings than Pune, while Pune has fewer cars compared to Gurgaon.

- **Dealer Name:**
  Dealers such as **Car Choice Exclusif**, **Car&Bike Superstore Pune**, and **Prestige Autoworld Pvt Ltd** have the highest number of listings, indicating their strong presence in the used car market.

```
sns.countplot(x = 'Owner', data = df).set_title('Number of cars by Owner Type')
```

```
Text(0.5, 1.0, 'Number of cars by Owner Type')
```



Owner Type Analysis

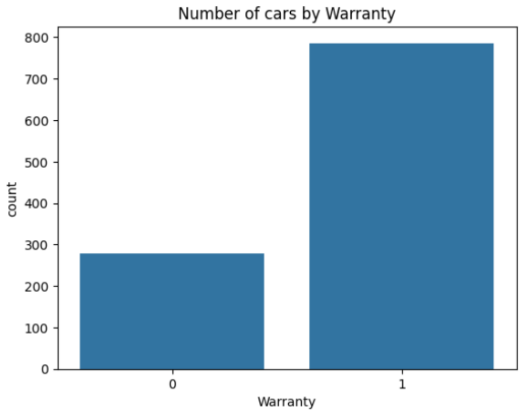
The **owner type** is an important factor influencing a car's condition and perceived value. From the graph, the majority of cars listed are **1st Owner vehicles**, followed by **2nd Owner cars**, which are significantly fewer.

Cars with **3rd and 4th ownership** are very limited in number. This distribution indicates that **1st Owner cars are more commonly available and generally preferred** in the used car market.

While fewer owners are often associated with better condition, the graph primarily reflects **availability and preference**, rather than directly proving higher resale value.

```
sns.countplot(x = 'Warranty', data = df).set_title('Number of cars by Warranty')
```

```
Text(0.5, 1.0, 'Number of cars by Warranty')
```



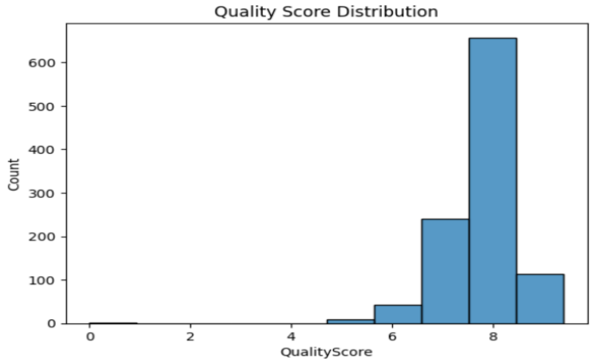
Warranty Analysis

This graph shows the number of used cars that come with a dealer warranty. It is observed that cars with a warranty are almost twice as many as those without a warranty, indicating that warranty-backed cars are more commonly listed in the dataset.

Since warranties provide additional assurance regarding vehicle condition and reduce risk for buyers, they can influence purchase decisions. However, the graph primarily reflects the availability of warranty-equipped cars rather than directly proving customer preference.

```
sns.histplot(x = 'QualityScore', data = df, bins = 10).set_title('Quality Score Distribution')
```

```
Text(0.5, 1.0, 'Quality Score Distribution')
```



Quality Score Analysis

Quality score is an important feature that influences a car's condition and perceived value. In the dataset, most cars have a quality score between **7 and 8**, indicating that a large proportion of vehicles are in reasonably good condition and have been evaluated before resale. Cars with higher quality scores are generally expected to be more reliable and may retain better value.

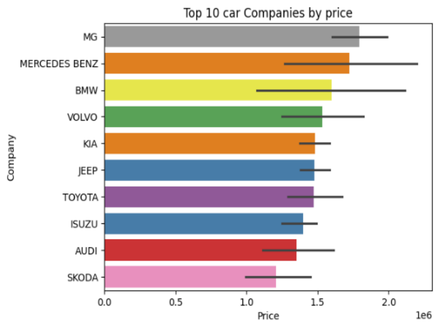
However, a small number of cars have a quality score below **5**, which may indicate poorer condition or older vehicles. Overall, the distribution shows that the dataset is dominated by cars of moderate to good quality.

Relationship Analysis

So far, the distribution of individual features has been analyzed to better understand the dataset. Next, the focus will be on examining the **relationship between car price and the independent variables** to identify the key factors influencing price prediction.

```
#Top 10 car companies by price
sns.barplot(y = 'Company', x = 'Price', data = df, order = df.groupby('Company')['Price'].mean().sort_values(ascending=False).iloc[:10].index, hue = 'Company', palette= 'Set1').set_title('Top 10 car Companies by price')
```

```
Text(0.5, 1.0, 'Top 10 car Companies by price')
```



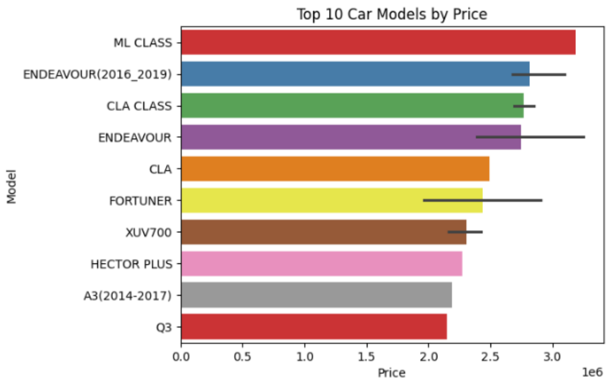
Car Company vs Price Analysis

This graph shows the top 10 car companies with the highest resale prices. **MG, Mercedes-Benz, and BMW** are among the top, followed by brands like **Volvo, Kia, Jeep, and Toyota**. These companies generally belong to the premium segment, which explains their higher resale values.

Interestingly, **Audi** has a comparatively lower resale price than other luxury brands in this dataset, possibly due to factors like model type or condition. Also, earlier assumptions about **Maruti Suzuki, Hyundai, Honda, Mahindra, and Tata** having high resale value are not correct—they appear more in number due to their affordability and popularity, not higher pricing.

```
sns.barplot(
    y='Model',
    x='Price',
    data=df,
    order=df.groupby('Model')['Price']
            .mean()
            .sort_values(ascending=False)
            .iloc[:10]
            .index,
    palette='Set1'
)

plt.title('Top 10 Car Models by Price')
plt.show()
```

```
/tmp/ipykernel_5607/948238688.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

  sns.barplot(
```



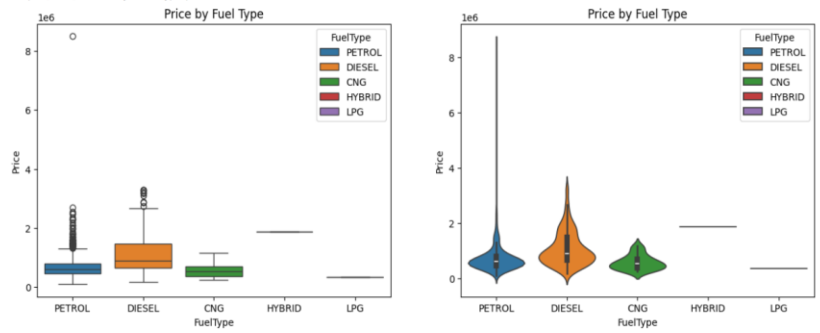
Car Model vs Price Analysis

This graph shows the relationship between car models and their resale value, and it follows a similar trend to the previous analysis. Models like **ML Class, Endeavour (2016–2019), and CLA Class** have the highest resale values, followed by **CLA, Fortuner, and XUV700**. These are generally premium or high-end models, which explains their higher prices.

Similar to the company-level analysis, **Audi A3** appears lower in the ranking compared to other premium models. Also, the earlier assumption that **Honda City, Swift, Baleno, Creta, and EcoSport** would have the highest resale values is not correct. These models appear more frequently in the dataset due to their **affordability and popularity**, not because of higher prices.

```
fig, ax = plt.subplots(1,2,figsize=(15,5))
sns.boxplot(x = 'FuelType', y = 'Price', data = df, ax = ax[0], hue = 'FuelType').set_title('Price by Fuel Type')
sns.violinplot(x = 'FuelType', y = 'Price', data = df, ax = ax[1], hue = 'FuelType').set_title('Price by Fuel Type')
```

```
Text(0.5, 1.0, 'Price by Fuel Type')
```

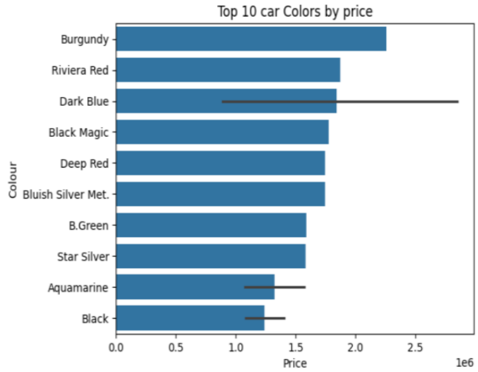


Fuel Type vs Price Analysis

The plots show the relationship between **fuel type** and car resale value. From the boxplot, **diesel cars** generally have a higher resale price compared to petrol, CNG, and LPG vehicles. The violin plot also indicates that diesel car prices are more concentrated in the **₹10–20 lakh** range compared to petrol cars.

This suggests that diesel vehicles tend to have **higher resale values**, possibly due to factors like engine efficiency and usage patterns. However, the plots mainly reflect **price distribution across fuel types** and do not directly prove customer demand, although petrol and diesel cars clearly dominate the dataset.

```
[52]
#Top 10 car colors by price
sns.barplot(y = 'Colour', x = 'Price', data = df, order = df.groupby('Colour')['Price'].mean().sort_values(ascending=False).iloc[:10].index).set_title('Top 10 car Colors by price')

Text(0.5, 1.0, 'Top 10 car Colors by price')
```

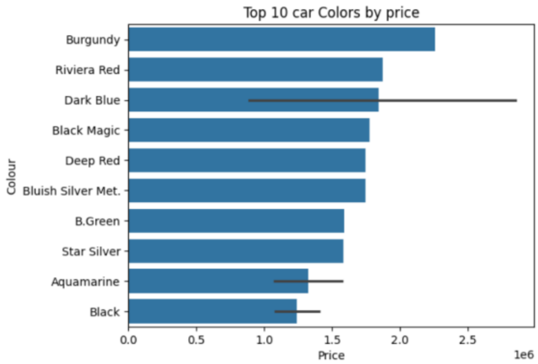


Car Color vs Price Analysis

Cars with colors like **Burgundy, Riviera Red, and Dark Blue** appear to have higher resale values compared to other colors. This may be influenced by the fact that such colors are often found in **premium or higher-end models**, rather than color alone driving the price.

Additionally, while these **exotic colors** may be associated with higher prices, they are less common in the dataset, indicating **lower availability** in the used car market. This suggests that color has some influence, but it is not a primary factor compared to features like brand, model, or age.


```
[52]
#Top 10 car colors by price
sns.barplot(y = 'Colour', x = 'Price', data = df, order = df.groupby('Colour')['Price'].mean().sort_values(ascending=False).iloc[:10].index).set_title('Top 10 car Colors by price')

Text(0.5, 1.0, 'Top 10 car Colors by price')
```

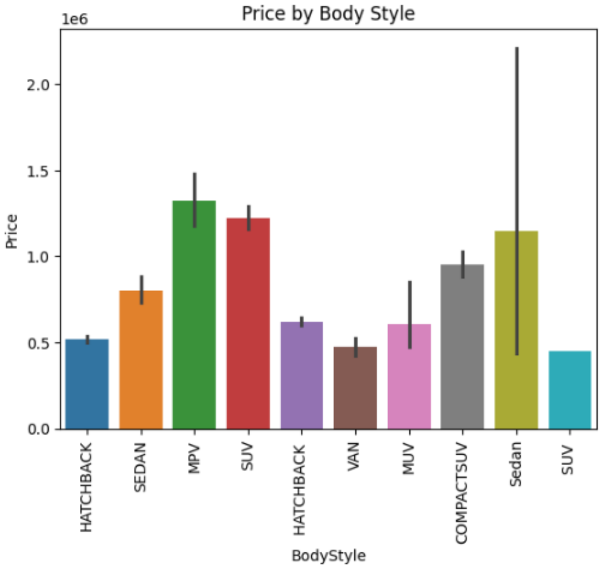
Odometer Reading vs Price

The scatter plot shows that most data points are concentrated below **100,000 km**, indicating that the majority of cars have relatively lower mileage. It is also observed that cars with **lower odometer readings tend to have higher resale prices**, while prices generally decrease as mileage increases.

This reflects the impact of usage on car value, where lower mileage is associated with better condition and higher pricing.

```
[54]
sns.barplot(x = 'BodyStyle', y = 'Price', data = df, hue = 'BodyStyle').set_title('Price by Body Style')
plt.xticks(rotation = 90)
```

```
([0, 1, 2, 3, 4, 5, 6, 7, 8, 9],
 [Text(0, 0, 'HATCHBACK'),
  Text(1, 0, 'SEDAN'),
  Text(2, 0, 'MPV'),
  Text(3, 0, 'SUV'),
  Text(4, 0, 'HATCHBACK '),
  Text(5, 0, 'VAN'),
  Text(6, 0, 'MUV'),
  Text(7, 0, 'COMPACTSUV'),
  Text(8, 0, 'Sedan'),
  Text(9, 0, 'SUV ')])
```



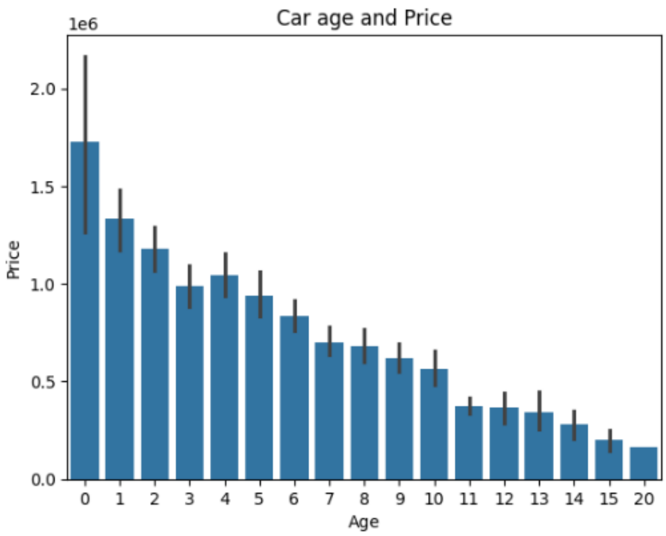
Body Style vs Price

The graph shows that **MPV, SUV, and Sedan** body styles have higher resale prices compared to others. This may be because these categories often include **larger or premium vehicles**.

Although **Hatchbacks** are more common in the dataset, they tend to have **lower resale prices**, indicating that higher availability does not necessarily correspond to higher value.

```
[55]
sns.barplot(x = 'Age', y = 'Price', data = df).set_title('Car age and Price')
```

```
Text(0.5, 1.0, 'Car age and Price')
```



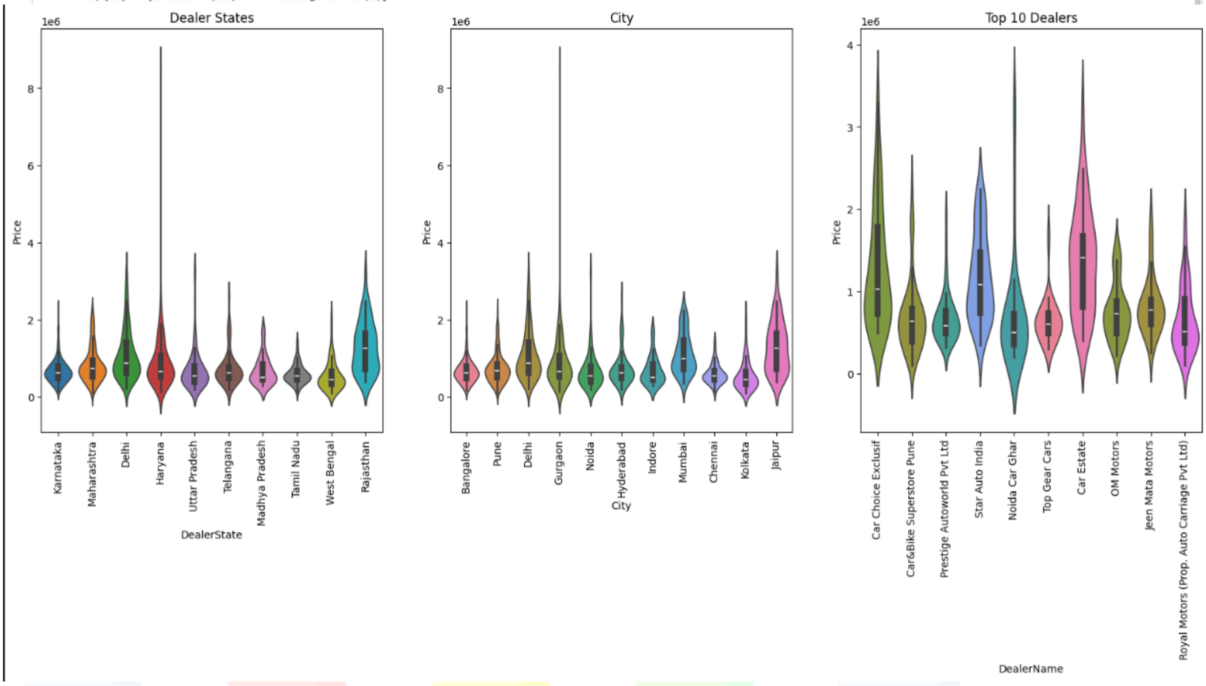
Car Age vs Price

The relationship between car age and price shows a clear trend. Cars with **lower age (especially less than 1–5 years)** have higher resale prices, while prices **decrease gradually as age increases**.

This confirms that depreciation plays a major role in determining car value, making age a key feature for price prediction.

```
[56]
fig, ax = plt.subplots(1,3,figsize=(20,7))

#Dealer State
sns.violinplot(x = 'DealerState', y = 'Price', data = df, ax = ax[0], hue = 'DealerState').set_title('Dealer States')
ax[0].set_xticklabels(ax[0].get_xticklabels(), rotation = 90)

#City
sns.violinplot(x = 'City',y = 'Price', data = df, ax = ax[1], hue = 'City').set_title('City')
ax[1].set_xticklabels(ax[1].get_xticklabels(), rotation = 90)

#top 10 dealers
sns.violinplot(x = 'DealerName',y = 'Price', data = df, order = df['DealerName'].value_counts().iloc[:10].index, ax = ax[2], hue = 'DealerName').set_title('Top 10 Dealers')
ax[2].set_xticklabels(ax[2].get_xticklabels(), rotation = 90)
```

```
/tmp/ipykernel_5607/7821321.py:5: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax[0].set_xticklabels(ax[0].get_xticklabels(), rotation = 90)
/tmp/ipykernel_5607/7821321.py:9: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax[1].set_xticklabels(ax[1].get_xticklabels(), rotation = 90)
/tmp/ipykernel_5607/7821321.py:13: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax[2].set_xticklabels(ax[2].get_xticklabels(), rotation = 90)
[Text(0, 0, 'Car Choice Exclusif'),
 Text(1, 0, 'Car&Bike Superstore Pune'),
 Text(2, 0, 'Prestige Autoworld Pvt Ltd'),
 Text(3, 0, 'Star Auto India'),
 Text(4, 0, 'Noida Car Ghar'),
 Text(5, 0, 'Top Gear Cars'),
 Text(6, 0, 'Car Estate'),
 Text(7, 0, 'OM Motors'),
 Text(8, 0, 'Jeen Mata Motors'),
 Text(9, 0, 'Royal Motors (Prop. Auto Carriage Pvt Ltd)')]
```



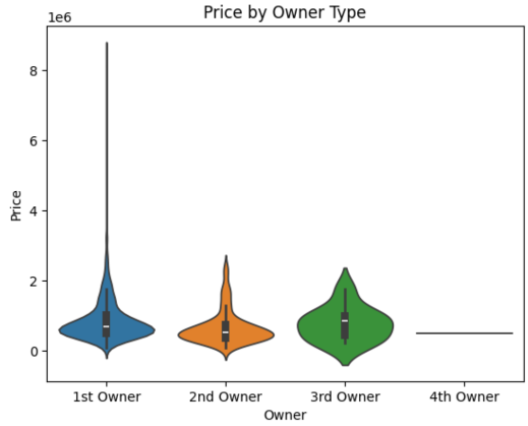
Location-Based Price Distribution

The graphs show price distribution across **states, cities, and dealers**. At the state level, **Rajasthan and Delhi** show higher price ranges. At the city level, **Jaipur, Mumbai, and Delhi** have higher-priced listings.

At the dealer level, dealers like **Car Estate, Star Auto India, and Car Choice** have higher price distributions. The presence of **outliers** in these plots (visible in violin plots) indicates that some listings have significantly higher prices, possibly due to premium models or specific conditions.

```
sns.violinplot(x = 'Owner', y = 'Price', data = df, hue = 'Owner').set_title('Price by Owner Type')
```

```
Text(0.5, 1.0, 'Price by Owner Type')
```



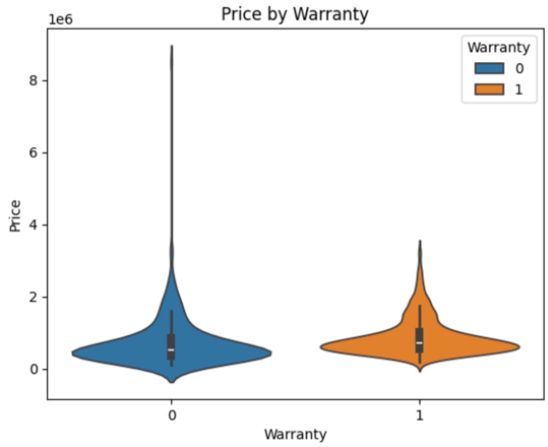
Car Owner Type vs Price

The graph shows the price distribution based on owner type. **1st Owner cars** generally have higher resale prices, which is expected as they are likely to be in better condition.

Interestingly, **3rd Owner cars** appear to have higher prices than 2nd Owner cars in some cases. This may be due to **outliers**, such as premium or niche vehicles, rather than a general trend.

```
sns.violinplot(x = 'Warranty', y = 'Price', data = df, hue = 'Warranty').set_title('Price by Warranty')
```

```
Text(0.5, 1.0, 'Price by Warranty')
```



Warranty vs Price

The violin plot shows that cars with a **warranty** tend to have slightly higher resale prices compared to those without a warranty.

This suggests that warranty may add value by providing **assurance about the vehicle's condition**, although the difference is not very large.

[59]

```
sns.scatterplot(x = 'QualityScore', y = 'Price', data = df).set_title('Quality Score and Price')
```

```
Text(0.5, 1.0, 'Quality Score and Price')
```

Quality Score vs Price

There is a clear trend showing that cars with a **quality score of 7 and above** have higher resale prices compared to those with lower scores.

This indicates that better vehicle condition, as reflected by higher quality scores, is associated with **higher pricing** in the used car market.

Data Preprocessing Part 2

Dropping column car model beacause, it has too many unique values and it will increase the dimensionality of the dataset.

[60]

```
df.drop('Model', axis = 1, inplace = True)
```

Label Encoding

[61]

```
#columns for label encoding
cols = df.select_dtypes(include=['object']).columns

from sklearn.preprocessing import LabelEncoder
#Label encoding object
le = LabelEncoder()

#label encoding for object type columns
for i in cols:
    le.fit(df[i])
    df[i] = le.transform(df[i])
    print(i, df[i].unique())
```

```
Company [12  7 19  5 13 21 11  6 17 16  9  4 20 10  1  3 18 14  0  8 22 15  2]
FuelType [4 1 0 2 5 3]
Colour [61 56 34  0  9 11 66 47 49 38 14 71 72 30 74 52 39 28 60  7 54 62 40 13
 20 70 63 12 24 23 35 26 29 15 31  1 68  4  8 73 22 44 57 65 42 50 32 64
 19 43 46 33 16 27 53 25 10 69 51 17  6 48 59 58  5  3 18 45 67 36 21 55
  2 37 75 41]
BodyStyle [1 5 3 6 2 9 4 0 8 7]
Owner [0 1 2 3]
DealerState [2 4 0 1 8 7 3 6 9 5]
DealerName [52 38  4  1 56 29  0 34 47 51 11 21  9 10 43 33  7 16  5 12 42 17 27 50
 45  6 20 36 23 41 32 31 18  2 48 15 54 40 55 13 49 25 35 46 24 14 44 19
 39 28 26  3 53 30  8 22 37]
City [ 0 10  2  3  9  4  5  8  1  7  6]
```

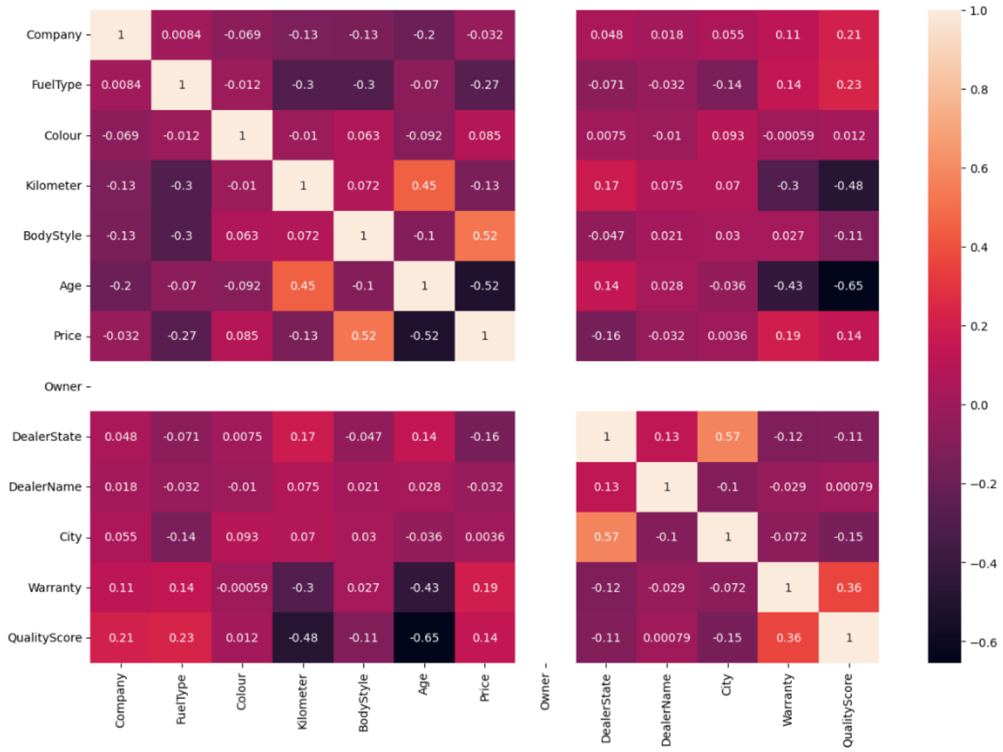
Outlier Removal

```
#Using IQRS to remove outliers

#columns for outlier removal
cols = df.select_dtypes(include=['int64','float64']).columns
Q1 = df[cols].quantile(0.25)
Q3 = df[cols].quantile(0.75)

IQR = Q3 - Q1

#Removing outliers
df = df[~((df[cols] < (Q1 - 1.5 * IQR)) |(df[cols] > (Q3 + 1.5 * IQR))).any(axis=1)]
```

Correlation Matrix Heatmap

```
plt.figure(figsize=(15,10))
sns.heatmap(df.corr(), annot=True)
```

```
<Axes: >
```



Train Test Split

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(df.drop('Price',axis=1), df['Price'], test_size=0.2, random_state=42)
```

Model Building

I will be using the following regression models:

- Decision Tree Regressor
- Random Forest Regressor
- Ridge Regressor

Decision Tree Regressor

```
from sklearn.tree import DecisionTreeRegressor
#Decision Tree Regressor Object
dtr = DecisionTreeRegressor()
```

Hyperparameter Tuning

[66]

```
from sklearn.model_selection import GridSearchCV

#parameters for grid search
para = {
    'max_depth' : [2,4,6,8],
    'min_samples_leaf' : [2,4,6,8],
    'min_samples_split' : [2,4,6,8],
    'random_state' : [0,42]
}

#Grid Search Object
grid = GridSearchCV(estimator=dtr, param_grid=para, cv=5, n_jobs=-1, verbose=2)

#Fitting the model
grid.fit(X_train, y_train)

#Best parameters
print(grid.best_params_)
```

```
Fitting 5 folds for each of 128 candidates, totalling 640 fits
{'max_depth': 6, 'min_samples_leaf': 4, 'min_samples_split': 2, 'random_state': 0}
```

[67]

```
#decision tree regressor with best parameters
dtr = DecisionTreeRegressor(max_depth=6, min_samples_leaf=2, min_samples_split=2, random_state=42)

#Fitting the model
dtr.fit(X_train, y_train)

#Training score
print(dtr.score(X_train, y_train))
```

```
0.7709748363868894
```

[68]

```
#Prediction
dtr_pred = dtr.predict(X_test)
```

Random Forest Regressor

[69]

```
from sklearn.ensemble import RandomForestRegressor
#Random Forest Regressor Object
rfr = RandomForestRegressor()
```

Hyperparameter Tuning

[70]

```
from sklearn.model_selection import GridSearchCV

#parameters for grid search
para = {
    'max_depth' : [2,4,6,8],
    'min_samples_leaf' : [2,4,6,8],
    'min_samples_split' : [2,4,6,8],
    'random_state' : [0,42]
}

#Grid Search Object
grid = GridSearchCV(estimator=rfr, param_grid=para, cv=5, n_jobs=-1, verbose=2)

#Fitting the model
grid.fit(X_train, y_train)

#Best parameters
print(grid.best_params_)
```

```
Fitting 5 folds for each of 128 candidates, totalling 640 fits
{'max_depth': 8, 'min_samples_leaf': 2, 'min_samples_split': 2, 'random_state': 0}
```

[71]

```
#Random Forest Regressor with best parameters
rfr = RandomForestRegressor(max_depth=8, min_samples_leaf=2, min_samples_split=2, random_state=0)

#Fitting the model
rfr.fit(X_train, y_train)

#Training score
print(rfr.score(X_train, y_train))
```

```
0.8922093892033637
```

[73]

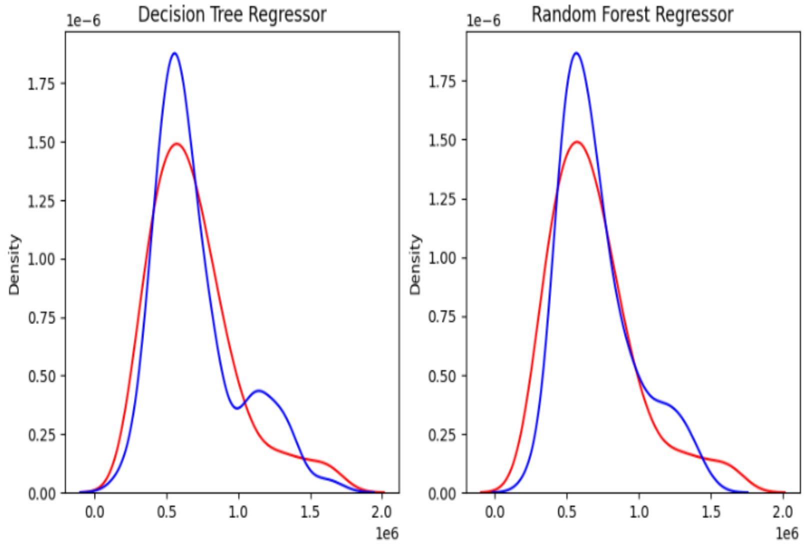
```
#Prediction
rfr_pred = rfr.predict(X_test)
```

Model Evaluation

Distribution Plot

```
fig,ax = plt.subplots(1,2,figsize=(10,5))

#decision tree regressor
sns.distplot(x = y_test, ax = ax[0], color = 'r', hist = False, label = 'Actual').set_title('Decision Tree Regressor')
sns.distplot(x = dtr_pred, ax = ax[0], color = 'b', hist = False, label = 'Predicted')

#random forest regressor
sns.distplot(x = y_test, ax = ax[1], color = 'r', hist = False, label = 'Actual').set_title('Random Forest Regressor')
sns.distplot(x = rfr_pred, ax = ax[1], color = 'b', hist = False, label = 'Predicted')
```

```
/tmp/ipykernel_5607/60301046.py:4: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `kdeplot` (an axes-level function for kernel density plots).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(x = y_test, ax = ax[0], color = 'r', hist = False, label = 'Actual').set_title('Decision Tree Regressor')
/tmp/ipykernel_5607/60301046.py:5: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `kdeplot` (an axes-level function for kernel density plots).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(x = dtr_pred, ax = ax[0], color = 'b', hist = False, label = 'Predicted')
/tmp/ipykernel_5607/60301046.py:8: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `kdeplot` (an axes-level function for kernel density plots).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(x = y_test, ax = ax[1], color = 'r', hist = False, label = 'Actual').set_title('Random Forest Regressor')
/tmp/ipykernel_5607/60301046.py:9: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `kdeplot` (an axes-level function for kernel density plots).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(x = rfr_pred, ax = ax[1], color = 'b', hist = False, label = 'Predicted')
<Axes: title={'center': 'Random Forest Regressor'}, ylabel='Density'>
```



Model Metrics

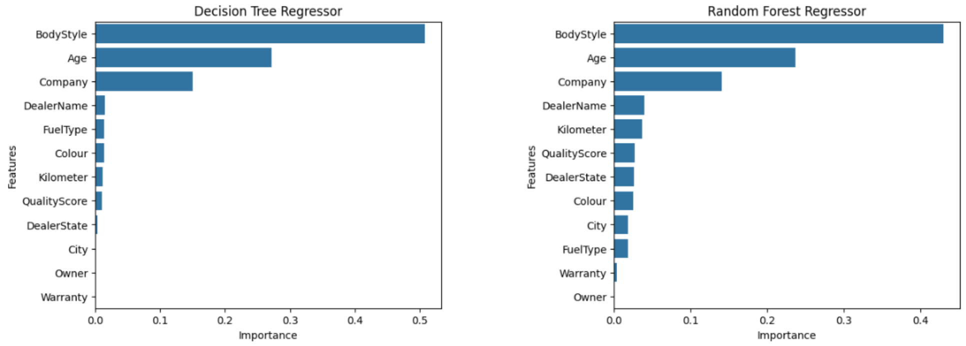
```
[75]
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

#Decision Tree Regressor
print('Decision Tree Regressor')
print('Mean Squared Error : ', mean_squared_error(y_test, dtr_pred))
print('Mean Absolute Error : ', mean_absolute_error(y_test, dtr_pred))
print('R2 Score : ', r2_score(y_test, dtr_pred))

#Random Forest Regressor
print('Random Forest Regressor')
print('Mean Squared Error : ', mean_squared_error(y_test, rfr_pred))
print('Mean Absolute Error : ', mean_absolute_error(y_test, rfr_pred))
print('R2 Score : ', r2_score(y_test, rfr_pred))
```

```
Decision Tree Regressor
Mean Squared Error :  45585786671.20511
Mean Absolute Error :  143988.5965345904
R2 Score :  0.5044410153036336
Random Forest Regressor
Mean Squared Error :  28952122030.700592
Mean Absolute Error :  122585.28783370154
R2 Score :  0.6852640867684735
```

Feature Importance

```
[76]
fig, ax = plt.subplots(1,2,figsize=(15, 5))
fig.subplots_adjust(wspace=0.5)

#Decision Tree Regressor
feature_df = pd.DataFrame({'Features':X_train.columns, 'Importance':dtr.feature_importances_})
feature_df.sort_values(by='Importance', ascending=False, inplace=True)
sns.barplot(x = 'Importance', y = 'Features', data = feature_df, ax = ax[0]).set_title('Decision Tree Regressor')

#Random Forest Regressor
feature_df = pd.DataFrame({'Features':X_train.columns, 'Importance':rfr.feature_importances_})
feature_df.sort_values(by='Importance', ascending=False, inplace=True)
sns.barplot(x = 'Importance', y = 'Features', data = feature_df, ax = ax[1]).set_title('Random Forest Regressor')
```

```
Text(0.5, 1.0, 'Random Forest Regressor')
```



Conclusion

From the exploratory data analysis, two key aspects of the used car market were identified: **price distribution and market presence**. The dataset shows a strong presence of **budget and mid-range cars**, indicating that lower-priced vehicles are more commonly available in the used car market. At the same time, premium brands such as **MG, Mercedes-Benz, BMW, Volvo, and Kia** have higher resale prices, while brands like **Maruti Suzuki, Hyundai, Honda, Mahindra, and Tata** dominate in terms of volume due to their affordability and popularity.

Most cars run on **petrol or diesel**, with diesel vehicles generally showing slightly higher prices. Common colors such as **white, grey, silver, and black** dominate the dataset, while some rare colors are associated with higher prices but appear less frequently. Factors like **lower mileage, newer age (less than 5 years), and better quality scores** are strongly associated with higher resale values. Additionally, **1st owner cars** and cars with **warranty** tend to have slightly higher prices, indicating better perceived condition and reliability.

Body styles such as **SUV, Sedan, and MPV** are associated with higher prices, while **Hatchbacks**, although more common, tend to have lower prices. Location also shows variation, with certain states, cities, and dealers having higher price distributions, along with some outliers due to premium listings.

For price prediction, **Decision Tree Regressor** and **Random Forest Regressor** models were implemented. Among them, the **Random Forest Regressor performed better**, providing more accurate predictions. Feature importance analysis revealed that **Car Age, Body Style, and Company** are among the most influential factors affecting car prices.

Overall, the project demonstrates how multiple factors such as **age, mileage, brand, condition, and features** collectively influence used car pricing, and how machine learning models can effectively predict these prices.